

# CHUNKWISE ALIGNERS FOR STREAMING SPEECH RECOGNITION

Wen Shen Teo<sup>1,2\*</sup>, Takafumi Moriya<sup>2\*</sup>, Masato Mimura<sup>2</sup>

<sup>1</sup>University of Electro-Communications, Japan; <sup>2</sup>NTT, Inc., Japan

## ABSTRACT

We propose the Chunkwise Aligner, a novel architecture for streaming automatic speech recognition (ASR). While the Transducer is the standard model for streaming ASR, its training is costly due to the need to compute all possible audio-label alignments. The recently introduced Aligner reduces this cost by discarding explicit alignments, but this modification makes it unsuitable for streaming. Our approach overcomes this limitation by dividing the audio into chunks and aligning each label to the leftmost frames of its chunk, whereas transitions between chunks are managed by a learned end-of-chunk probability. Experiments show that the Chunkwise Aligner not only matches the Transducer’s accuracy in both offline and streaming scenarios, but also offers superior training and decoding efficiencies.

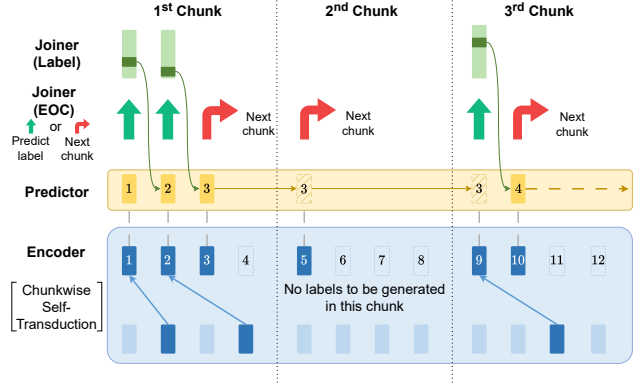
**Index Terms**— speech recognition, aligner, streaming, chunkwise aligner

## 1. INTRODUCTION

State-of-the-art automatic speech recognition (ASR) systems are largely built on two dominant architectures: the Attention-based Encoder-Decoder (AED) [1, 2] and the Transducer [3]. These architectures each offer distinct advantages. In this paper, we examine and compare them in terms of decoding speed, recognition accuracy, and streaming compatibility.

The AED is a powerful model trained in a label-synchronous manner with a simple cross-entropy objective, which makes its training process conceptually straightforward and efficient. However, this simplicity comes at the cost of being inherently non-streaming, since its decoder requires cross-attention over the entire audio sequence to predict each label. In contrast, and more relevant to our goal of streaming ASR, the Transducer is a frame-synchronous model that naturally supports real-time applications [4]. This capability, however, relies on a more complex objective function, which requires a computationally expensive dynamic programming step to marginalize over all possible alignments between input audio frames and output labels.

A recent model, the Aligner [5], seeks to combine the strengths of both the Transducer and the AED. It adopts the Transducer architecture but removes the blank transition probability, and introduces a process called “self-transduction”, in which the self-attention modules in the encoder learn to sequentially align all output labels to the leftmost frames of the input sequence. Through this mechanism, the Aligner enables AED-like, label-synchronous training with a simple cross-entropy loss, whereas its decoding utilizes a Transducer-like, fast and lightweight autoregressive component. However, by collapsing information to the beginning of the sequence, the model discards the local temporal alignment between the audio and the labels,



**Fig. 1.** Schematic diagram of our proposed Chunkwise Aligner. It predicts label probabilities and the EOC probability separately. Chunkwise self-transduction performed by the encoder aligns label information (dense blue blocks) to the leftmost frames of each chunk. An EOC probability exceeding a predefined threshold advances the model into the next chunk, even if no labels were predicted (as is the case of the 2nd chunk).

which is essential for streaming recognition. Moreover, the Aligner is not robust to unseen speech lengths during inference and requires data concatenation to generalize across input lengths, thereby increasing its training cost.

To address this limitation, we propose the *Chunkwise Aligner*, a novel architecture that adapts the self-transduction mechanism for streaming ASR. As illustrated in Fig. 1, our model partitions the input audio into fixed-size chunks. Using a given alignment, it performs self-transduction locally by sequentially mapping the labels corresponding to each chunk to the leftmost frames within that chunk. To handle transitions between chunks, we introduce a learnable end-of-chunk (EOC) probability that indicates when to advance. Upon detecting an EOC, the hypotheses and decoder states of the current chunk are carried forward to the next, enabling the cycle to repeat for continuous audio streams. Experimental results show that our Chunkwise Aligner not only surpasses the Aligner, but achieves both offline and streaming ASR performances on par with that of the Transducer, while significantly reducing computational training costs and improving decoding speed.

## 2. PRELIMINARIES

The Transducer, Aligner, and our Chunkwise Aligner are all built upon a common encoder-predictor-joiner architecture. This section will first describe this common framework and then detail the joiner formulation, which differs between the Transducer and Aligner.

### 2.1. Model architecture

The architecture consists of three main components: an encoder, a predictor, and a joiner. The processing of each component is de-

\* Equal contribution. This work was conducted during the first author’s internship at NTT, Inc.

scribed as follows:

- **Encoder** includes a self-attention module (e.g., Conformer [6]) and transforms an input acoustic feature sequence  $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_T]$  of length  $T$  into a  $D'$ -dimensional higher-level representation  $\mathbf{H}^{\text{enc}} = [\mathbf{h}_1^{\text{enc}}, \dots, \mathbf{h}_T^{\text{enc}}] \in \mathbb{R}^{T \times D'}$ .
- **Predictor** processes the previously emitted non-blank labels  $Y = [y_1, \dots, y_U]$  of length  $U$ , where  $y_u \in \mathcal{Y}$ , to autoregressively produce a sequence of  $D''$ -dimensional contextual embeddings  $\mathbf{H}^{\text{pred}} = [\mathbf{h}_1^{\text{pred}}, \dots, \mathbf{h}_U^{\text{pred}}] \in \mathbb{R}^{U \times D''}$ .  $\mathcal{Y}$  is the token set that consists of  $V$  tokens.
- **Joiner** predicts the next label probabilities (additionally including a transition probability for the Transducer and Chunkwise Aligner) using  $\mathbf{H}^{\text{enc}}$  and  $\mathbf{H}^{\text{pred}}$ .

## 2.2. Joiner formulations

### 2.2.1. Transducer

The Transducer [3, 7] learns the mapping between sequences of different lengths through the introduction of blank symbol. Thus, the joiner computation must consider all possible alignments using the outputs of encoder and predictor. In this work, we employ the Hybrid Autoregressive Transducer (HAT) [7], which produces the blank probability  $\hat{y}_{t,u}^{\text{blank}} \in (0, 1)^1$  and the label probabilities  $\hat{y}_{t,u}^{\text{label}} \in (0, 1)^V$  separately. Here, we describe the details of the joiner processing in the HAT as follows:

$$\mathbf{h}_{t,u}^{\text{joiner}} = \tanh(\mathbf{W}^{\text{enc}} \mathbf{h}_t^{\text{enc}} + \mathbf{W}^{\text{pred}} \mathbf{h}_u^{\text{pred}} + \mathbf{b}), \quad (1)$$

$$\hat{y}_{t,u}^{\text{blank}} = \text{Sigmoid}(\mathbf{w}^{\text{blank}} \bullet \mathbf{h}_{t,u}^{\text{joiner}} + b^{\text{blank}}), \quad (2)$$

$$\hat{y}_{t,u}^{\text{label}} = \text{Softmax}(\mathbf{W}^{\text{label}} \mathbf{h}_{t,u}^{\text{joiner}} + \mathbf{b}^{\text{label}}), \quad (3)$$

$$\hat{\mathbf{y}}_{t,u} = \left[ \hat{y}_{t,u}^{\text{blank}}, (1 - \hat{y}_{t,u}^{\text{blank}}) \cdot \hat{\mathbf{y}}_{t,u}^{\text{label}^\top} \right]^\top \in (0, 1)^{V+1}, \quad (4)$$

where  $\{\mathbf{W}^{\text{enc}} \in \mathbb{R}^{D \times D'}, \mathbf{W}^{\text{pred}} \in \mathbb{R}^{D \times D''}, \mathbf{b} \in \mathbb{R}^D\}$  is the parameter set of the joiner space, which produces  $\mathbf{h}_{t,u}^{\text{joiner}}$  via the hyperbolic tangent function  $\tanh(\cdot)$ .  $\hat{y}_{t,u}^{\text{blank}}$  is computed using weight vector  $\mathbf{w}^{\text{blank}} \in \mathbb{R}^D$  and bias term  $b^{\text{blank}} \in \mathbb{R}^1$ , where  $\bullet$  and  $\text{Sigmoid}(\cdot)$  denote the dot-product and sigmoid operation, respectively.  $\{\mathbf{W}^{\text{label}} \in \mathbb{R}^{V \times D}, \mathbf{b}^{\text{label}} \in \mathbb{R}^V\}$  are the parameters of the linear output layer, which yields  $\hat{\mathbf{y}}_{t,u}^{\text{label}}$  via the softmax function  $\text{Softmax}(\cdot)$ . Consequently, during training, the joiner computes a memory-intensive three-dimensional grid of predictions with shape  $T \times U \times (V + 1)$ . The Transducer loss is defined as the full-sum loss and can be computed efficiently using the forward-backward algorithm [3].

### 2.2.2. Aligner

The Aligner [5] simplifies the joiner computation by enforcing a one-to-one mapping. It performs “self-transduction” using the self-attention module within the encoder, without employing cross-attention as in the AED. The self-transduction internally reorders acoustic information so that the leftmost  $U$  frames of  $\mathbf{H}^{\text{enc}}$  are aligned with the  $U$  labels. Therefore, the joiner of the Aligner can easily compute the output probabilities  $\hat{\mathbf{y}}_u^{\text{label}} \in (0, 1)^V$  as follows:

$$\mathbf{h}_u^{\text{joiner}} = \tanh(\mathbf{W}^{\text{enc}} \mathbf{h}_u^{\text{enc}} + \mathbf{W}^{\text{pred}} \mathbf{h}_u^{\text{pred}} + \mathbf{b}), \quad (5)$$

$$\hat{\mathbf{y}}_u^{\text{label}} = \text{Softmax}(\mathbf{W}^{\text{label}} \mathbf{h}_u^{\text{joiner}} + \mathbf{b}^{\text{label}}). \quad (6)$$

Self-transduction enforces a fixed alignment, so the blank symbol for transitions is not required, and the joiner may compute a distribution over only the  $V$  text tokens. The dense Transducer grid is

replaced with a two-dimensional  $U \times V$  tensor, which is optimized by a cross-entropy loss. However, this full-context reordering discards the original temporal correspondence between the audio and labels, making the Aligner inherently non-streaming.

## 3. PROPOSED CHUNKWISE ALIGNER

Our Chunkwise Aligner performs chunkwise decoding, and its processing is illustrated in Fig. 1. The encoder output sequence  $\mathbf{H}^{\text{enc}}$  is segmented into  $N$  chunks of length  $L_c$ , denoted as  $(\mathbf{H}_1^{\text{enc}}, \dots, \mathbf{H}_N^{\text{enc}})$ , where  $\mathbf{H}_n^{\text{enc}} = [\mathbf{h}_{(n-1) \times L_c + 1}^{\text{enc}}, \dots, \mathbf{h}_{n \times L_c}^{\text{enc}}]^\top$  represents the  $n$ -th input chunk. This enables the Chunkwise Aligner to operate in streaming ASR. The predictor output sequence  $\mathbf{H}^{\text{pred}}$  is also divided into  $N$  chunks  $(\mathbf{H}_1^{\text{pred}}, \dots, \mathbf{H}_N^{\text{pred}})$  to match the encoder output chunks. Labels are assigned to chunks via forced alignment (similar to [8]), thus the number of labels in each chunk varies. Hence, the sequence of predictor outputs belonging to the  $n$ -th chunk is defined as  $\mathbf{H}_n^{\text{pred}} = [\mathbf{h}_{u_n}^{\text{pred}}]_{u_n \in (1, \dots, U_n)}^\top$  where  $U_n$  denotes the total number of labels assigned to the  $n$ -th chunk, and  $u_n$  denotes the assigned position of the label within its chunk. Whether to transition to the next chunk is determined by the end-of-chunk (EOC) probability, instead of the blank probability in the HAT joiner as described in Section 2.2.1. The joiner of the Chunkwise Aligner has the same number of parameters as the HAT joiner and is defined as follows:

$$\mathbf{h}_{u_n}^{\text{joiner}} = \tanh(\mathbf{W}^{\text{enc}} \mathbf{h}_{(n-1) \times L_c + u_n}^{\text{enc}} + \mathbf{W}^{\text{pred}} \mathbf{h}_{u_n}^{\text{pred}} + \mathbf{b}), \quad (7)$$

$$\hat{y}_{u_n}^{\text{eoc}} = \text{Sigmoid}(\mathbf{w}^{\text{eoc}} \bullet \mathbf{h}_{u_n}^{\text{joiner}} + b^{\text{eoc}}), \quad (8)$$

$$\hat{\mathbf{y}}_{u_n}^{\text{label}} = \text{Softmax}(\mathbf{W}^{\text{label}} \mathbf{h}_{u_n}^{\text{joiner}} + \mathbf{b}^{\text{label}}), \quad (9)$$

where  $\hat{y}_{u_n}^{\text{eoc}} \in (0, 1)^1$  and  $\hat{\mathbf{y}}_{u_n}^{\text{label}} \in (0, 1)^V$  are the  $u_n$ -th EOC probability and label probabilities in the  $n$ -th chunk, respectively. Thus, the self-attention modules in the Chunkwise Aligner’s encoder are trained to perform the self-transduction mechanism of the Aligner in a chunkwise manner, i.e. “chunkwise self-transduction”.

### 3.1. Training objectives

The tensor format of the label probabilities is the same as that of the Aligner,  $U \times V$ , and they are optimized using a standard cross-entropy loss,  $\mathcal{L}_{\text{label}}$ . However, as shown in Fig. 1, the EOC probabilities require one additional entry for each chunk, resulting in a prediction grid of  $(U + N) \times 1$ , which corresponds to the label length  $U$  plus the chunk size  $N$ . For each chunk  $n$ , the  $U_n$ -th predictor output is paired with the  $(U_n + 1)$ -th encoder output to compute the final EOC probability of that chunk. To train the EOC branch in the Chunkwise Aligner’s joiner, we prepare  $U + N$  target labels such that only the final position in each chunk is set to 1, while all others are set to 0. The resulting EOC probabilities are then optimized with a binary cross-entropy loss,  $\mathcal{L}_{\text{eoc}}$ . The final total loss,  $\mathcal{L}_{\text{total}}$ , is then a simple sum of these two components:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{label}} + \mathcal{L}_{\text{eoc}}. \quad (10)$$

The prediction grids sum to a total size of  $U \times (V + 1 + \frac{N}{U})$ , providing huge computational improvements compared to the Transducer,  $T \times U \times (V + 1)$ . Therefore, the Chunkwise Aligner drastically reduces memory and computational load during training, while retaining the streaming capability that the original Aligner lacks.

**Algorithm 1** Beam search decoding for the Chunkwise Aligner

---

```

1: function BEAMSEARCH( $H^{\text{enc}}, L_c, \text{beam\_size}, \tau$ )
2:   Input: Encoder output, Chunk size, Beam size, Threshold
3:   Output: Most likely hypotheses
4:    $B \leftarrow \{(\text{<eos>}, 1, s_0)\}$ ;  $N \leftarrow \lceil |H^{\text{enc}}|/L_c \rceil$ ;  $F \leftarrow \{\}$ 
5:   for  $n = 1, \dots, N$  do ▷ Chunkwise processing
6:      $H_n^{\text{enc}} \leftarrow \text{ChunkGenerator}(H^{\text{enc}}, L_c, n)$ ;  $C \leftarrow \{\}$ 
7:     for  $h_t^{\text{enc}} \in H_n^{\text{enc}}$  do
8:        $A \leftarrow \{\}$ 
9:       if  $B$  is empty then
10:        break
11:       end if
12:       for  $(y, \delta_{u-1}, s_{u-1}) \in B$  do
13:          $h_{u-1}^{\text{pred}}, s_u \leftarrow \text{Predictor}(y_{u-1}, s_{u-1})$ 
14:          $h_u^{\text{joiner}} \leftarrow \text{Joiner}(h_t^{\text{enc}}, h_{u-1}^{\text{pred}})$  ▷ Eq. (7)
15:          $\hat{y}_u^{\text{eoc}} \leftarrow \text{JoinerEOC}(h_u^{\text{joiner}})$  ▷ Eq. (8)
16:         if  $\hat{y}_u^{\text{eoc}} > \tau$  then
17:            $C \leftarrow C \cup \{(y, \delta_{u-1} * \hat{y}_u^{\text{eoc}}, s_{u-1})\}$ 
18:           continue
19:         end if
20:          $\hat{y}_u^{\text{label}} \leftarrow \text{JoinerLabel}(h_u^{\text{joiner}})$  ▷ Eq. (9)
21:         for  $k \in \mathcal{Y}$  do
22:            $\delta_u \leftarrow \delta_{u-1} * \hat{y}_u^{\text{label}}[k] * (1 - \hat{y}_u^{\text{eoc}})$ 
23:           if  $k$  is <eos> then
24:              $F \leftarrow F \cup \{(y + k, \delta_u, s_u)\}$ 
25:           else
26:              $A \leftarrow A \cup \{(y + k, \delta_u, s_u)\}$ 
27:           end if
28:         end for
29:       end for
30:        $B \leftarrow \text{PruneHyps}(A, \text{beam\_size})$ 
31:     end for
32:      $B \leftarrow \text{PruneHyps}(B \cup C, \text{beam\_size})$ 
33:   end for
34:   return Sorted( $F$ )
35: end function

```

---

**3.2. Decoding strategy**

Algorithm 1 shows the chunkwise decoding process for the Chunkwise Aligner. We prepare tentative and final hypothesis sets,  $B$  and  $F$ , respectively (Line 4).  $B = \{y, \delta, s\}$  is initialized with the start-of-sentence label <eos>, a score of 1, and the predictor state  $s_0$ . From Line 5, chunkwise decoding begins: the encoder output sequence  $H^{\text{enc}}$  is segmented into  $N$  chunks, and an empty set  $C$  is initialized to store hypotheses that have predicted a chunk transition (Line 6). The inner loop (Line 7) performs a framewise beam search within the current chunk,  $H_n^{\text{enc}}$ , until the end of the current chunk is reached or the tentative hypothesis set  $B$  becomes empty. From Line 12, for each hypothesis in the tentative hypothesis set  $B$ , the predictor and joiner computations are performed (Lines 13, 14, 15, and 20), and the resulting hypotheses are appended to one of the hypothesis sets  $C$ ,  $F$ , or  $A$ . Note that during inference, the ground truth transcript is unknown, and we cannot calculate which chunk contains the  $u$ -th label beforehand. Instead, if  $\hat{y}_u^{\text{eoc}}$  exceeds a threshold of  $\tau = 0.5$ <sup>1</sup>, we terminate the search for this chunk, and append that hypothesis to  $C$  (Line 17). When computing the label probabilities  $\hat{y}_u^{\text{label}}$  (Line 21), if the label  $k$  is the end-of-sentence label <eos>, the corresponding hypothesis is appended to  $F$ ; otherwise, it is appended to  $B$ . PrunedHyps( $\cdot, \cdot$ ) in Lines 30 and 32 sorts the

<sup>1</sup>Preliminary experiments showed that varying the threshold  $\tau$  did not yield a significant impact on recognition performance.

input hypothesis set by score and returns the top hypotheses, up to the beam size. If the decoding step reaches the end of the final chunk  $N$ , the hypotheses  $F$ , sorted by score, are returned (Line 34).

Compared to the Transducer, our Chunkwise Aligner enables a simple, label-synchronous-like algorithm that ensures decoding steps proportional to the hypothesis label length  $U$  instead of the acoustic frame length  $T$ . Our decoding algorithm can reduce the computation of label probabilities  $\hat{y}_u^{\text{label}}$ , which are computationally expensive, by thresholding  $\tau$  (Line 17). If all hypotheses are stored in  $C$  (Line 10), it is no longer necessary to compute up to the last frame of the chunk, leading to a further substantial reduction in computational cost. Our streaming Chunkwise Aligner does not require cross-attention and achieves faster decoding than chunkwise AED-based models [9, 10, 11, 12], which are complex in both training and inference. Moreover, this also enables Chunkwise Aligner decoding to be faster than Aligner decoding, because the latter, like AED, requires a maximum expected length of the hypothesis, leading to more redundant computation [13, 14].

**4. EXPERIMENTAL EVALUATIONS****4.1. Data**

We evaluated our proposed Chunkwise Aligner models on LibriSpeech [15] and Corpus of Spontaneous Japanese (CSJ) [16] using ESPnet [14]. The input feature was an 80-dimensional log Mel-filterbank extracted with a 25ms window and a 10ms stride. Augmentation methods [17, 18, 19] were applied during training. We adopted a word-piece tokenizer [20] with a vocabulary size of 1,000 for LibriSpeech, and a character-based tokenizer resulting in a vocabulary size of 3,262 for CSJ. Ground truth alignments for Chunkwise Aligner training were generated using the Montreal Forced Aligner (MFA) [21].

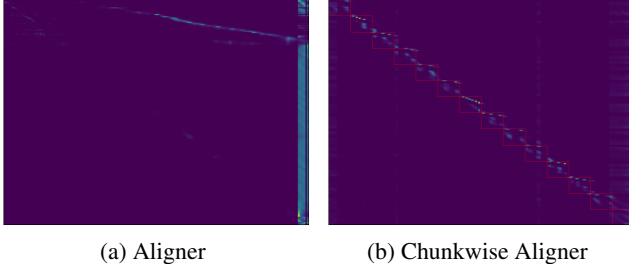
**4.2. System Configuration**

All models were built on the same encoder architecture. We used 17 Conformer blocks with the same setup as Conformer-L [6], comprising approximately 110M parameters. An embedder of a two-layer 2-dimensional convolutional neural network (CNN) with 256 filters and a frame reduction rate of 4 was used before the Conformer blocks. The kernel size of 31 and the batch normalization within the convolution module were replaced with a kernel size of 15 and layer normalization. In streaming mode, both the current and history encoder chunk sizes were set to 15, which when considering the subsampling nature of our embedder, results in an algorithmic latency of 600ms. Depthwise convolution is replaced with a causal variant for the streaming encoder [22]. The predictor consisted of a 640-dimensional LSTM [23]. The decoding chunk size of Chunkwise Aligners was fixed at 15 in both offline and streaming modes. Additionally, for comparison, we built AED models with four Transducer decoder blocks, following [5].

The parameters of all models were initialized with random values. But, as in [8], only the encoder was initialized with parameters pretrained with Connectionist Temporal Classification (CTC) [24] when using CTC alignments for Chunkwise Aligner training. This pretrained encoder also included an Inter-CTC loss [25] at the 10th layer, from which the alignments were generated. We used the Adam optimizer [26] with a warm-up phase of 25k steps and a peak learning rate of  $1.5e-3$ . Training was conducted for a total of 100 epochs. A beam size of 8 during inference was used in all models. We evaluated performance in terms of word error rate (WER) for LibriSpeech and character error rate (CER) for CSJ. We also measured the real-time factor (RTF: *decoding time / data time*) for each offline model, tested on an Intel Xeon Gold 6430 3.4GHz CPU.

**Table 1.** WERs [%] and RTFs for **Offline ASR** on the LibriSpeech test sets. “Alignment type” shows the forced alignment method used.

|                           | Alignment type | WER   |       | RTF  |
|---------------------------|----------------|-------|-------|------|
|                           |                | clean | other |      |
| Aligner (+DataConcat) [5] | -              | 2.3   | 5.1   | N/A  |
| Transducer                | -              | 2.2   | 4.9   | 0.30 |
| CTC                       | -              | 2.7   | 6.7   | 0.09 |
| AED (+DataConcat)         | -              | 2.4   | 5.4   | 0.49 |
| Aligner (+DataConcat)     | -              | 2.4   | 5.7   | 0.18 |
| Chunkwise Aligner         | ground-truth   | 2.2   | 5.0   | 0.12 |
|                           | offline CTC    | 2.2   | 5.0   | 0.12 |



**Fig. 2.** Self-attention weights from a single head at the 16th layer of (a) the Aligner and (b) the Chunkwise Aligner encoders performing audio-to-text alignment. The red boxes in (b) mark the chunk boundaries along the diagonal. These figures are generated from the utterance “1688-142285-0006” (34 tokens).

#### 4.3. Results on LibriSpeech

First, we present the offline ASR results on the LibriSpeech test sets in Table 1, where the solid line separates the baselines (top) and our proposed models (bottom). The WERs for CTC were obtained from the pretrained model used for alignment generation. When training the AED and Aligner models, we randomly concatenated two utterances with a probability of 15% (“+DataConcat”). This is to improve performance on the longer utterances in the test sets, as demonstrated in [5]. Our reproduced Aligner model (+DataConcat) was not enough to achieve results reported in the literature [5]. We attribute this difficulty to the more challenging task of global self-transduction as the audio length increases, requiring the encoder to memorize the positions of the leftmost frames and remap acoustic information through the self-attention space. Unlike the Aligner, our Chunkwise Aligner does not need data concatenation.

Comparing against the Transducer highlights two advantages of our method. Despite its simple cross-entropy training, our Chunkwise Aligner achieved comparable results to the memory-intensive Transducer. And, by virtue of our chunk transition mechanism with the EOC probability, it activates its predictor and joiner label-synchronously, leading to an inference time 2.6 times shorter than the Transducer. For the offline scenario, we observe no performance difference between training with alignments generated from ground-truth and that from a pretrained CTC.

We plotted the self-attention weights of the best offline Aligner and Chunkwise Aligner models in Fig. 2. Consistent with the findings in [5], we find that the Aligner maps all labels sequentially to the leftmost frames of the utterance. Meanwhile, our Chunkwise Aligner aligns labels on a chunkwise basis. This shows that chunkwise self-transduction is a simpler objective because it maps the alignment over shorter distances, leading to robust generalization to utterances with unseen lengths and lower WERs.

**Table 2.** WERs [%] for **Streaming ASR** on the LibriSpeech test sets. “Delay” is the delay duration applied to the forced alignments.

|                   | Alignment type | Delay | WER   |       |
|-------------------|----------------|-------|-------|-------|
|                   |                |       | clean | other |
| Transducer        | -              | -     | 3.1   | 7.6   |
| CTC               | -              | -     | 4.1   | 10.8  |
| Chunkwise Aligner | ground-truth   | 0ms   | 3.9   | 9.5   |
|                   | ground-truth   | 160ms | 3.5   | 8.5   |
|                   | ground-truth   | 320ms | 3.2   | 7.9   |
|                   | ground-truth   | 480ms | 3.4   | 8.3   |
|                   | streaming CTC  | 0ms   | 3.6   | 9.0   |

**Table 3.** CERs [%] and RTFs for the CSJ test sets. Chunkwise Aligner models were trained using CTC alignments with no delay.

|                   | Offline |     |     |      | Streaming |     |     |
|-------------------|---------|-----|-----|------|-----------|-----|-----|
|                   | E1      | E2  | E3  | RTF  | E1        | E2  | E3  |
| Transducer        | 4.1     | 3.0 | 3.4 | 0.30 | 5.1       | 3.9 | 4.1 |
| CTC               | 4.2     | 3.1 | 3.6 | 0.10 | 5.3       | 4.2 | 4.4 |
| AED               | 3.9     | 2.9 | 3.4 | 0.55 | N/A       |     |     |
| Aligner           | 4.2     | 3.2 | 3.6 | 0.22 | N/A       |     |     |
| Chunkwise Aligner | 4.0     | 2.9 | 3.4 | 0.16 | 5.1       | 3.9 | 4.1 |

Chunkwise self-transduction not only improves the performance of an offline Chunkwise Aligner, but unlike the Aligner, it also allows streaming ASR when a streaming encoder is adopted. Table 2 presents our streaming ASR results. Compared to offline models, streaming ASR models commonly delay label emission. Therefore, the Chunkwise Aligner, which relies on forced alignments during training, may suffer from performance degradation due to mismatches in timing [27, 28]. We investigated how the WERs are affected when the timestamps of ground truth forced alignments are delayed. As can be seen, the best performance was achieved with a 320ms delay, yielding WERs comparable to the streaming Transducer. Contrary to the offline scenario, we observed that using CTC alignments worsened performance. This is discussed in Section 4.5.

#### 4.4. Results on CSJ

Shown in Table 3, our results for CSJ mirror those observed for LibriSpeech. The Chunkwise Aligner again achieved CERs comparable to the Transducer in both offline and streaming settings, while decoding 1.8 times faster in the offline scenario. This finding demonstrates the cross-lingual applicability of our proposed method.

#### 4.5. Discussion

Our results show that our method’s performance is dependent on the alignment used during training. We hypothesize that the degraded WERs on LibriSpeech when trained on CTC alignments is due to the subpar WERs of the streaming CTC model, from which the Chunkwise Aligner was initialized. This hypothesis is supported by the strong results on CSJ by both the streaming CTC and Chunkwise Aligner models. Improving performance without relying on high quality alignments remains a direction for future work.

### 5. CONCLUSION

We proposed the Chunkwise Aligner, which improves the Aligner to support streaming through chunkwise processing. When compared to the Transducer, our method not only reduces training costs by relying entirely on cross-entropy training, but also decodes faster while maintaining comparable accuracy. As future work, we plan to explore alignment-free training frameworks and investigate whether chunkwise aligners can work in attention-free encoders [29, 30].

## 6. REFERENCES

- [1] Jan K Chorowski, Dzmitry Bahdanau, Dmitriy Serdyuk, Kyunghyun Cho, and Yoshua Bengio, “Attention-based Models for Speech Recognition,” in *Advances in NIPS*, 2015.
- [2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin, “Attention is All You Need,” *Advances in NIPS*, 2017.
- [3] Alex Graves, “Sequence Transduction with Recurrent Neural Networks,” in *Proc. of ICML*, 2012.
- [4] Tara N. Sainath, Yanzhang He, Arun Narayanan, Rami Botros, Ruoming Pang, David Rybach, Cyril Allauzen, Ehsan Variani, James Qin, Quoc-Nam Le-The, Shuo yiin Chang, Bo Li, Anmol Gulati, Jiahui Yu, Chung-Cheng Chiu, Diamantino Casero, Wei Li, Qiao Liang, and Pat Rondon, “An Efficient Streaming Non-Recurrent On-Device End-to-End Model with Improvements to Rare-Word Modeling,” in *Proc. of INTERSPEECH*, 2021, pp. 1777–1781.
- [5] Adam Stooke, Rohit Prabhavalkar, Khe Sim, and Pedro Moreno Mengibar, “Aligner-Encoders: Self-Attention Transformers Can Be Self-Transducers,” in *Advances in NeurIPS*, pp. 100318–100340, 2024.
- [6] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al., “Conformer: Convolution-augmented Transformer for Speech Recognition,” in *Proc. of INTERSPEECH*, 2020, pp. 5036–5040.
- [7] Ehsan Variani, David Rybach, Cyril Allauzen, and Michael Riley, “Hybrid Autoregressive Transducer (HAT),” in *Proc. of ICASSP*, 2020, pp. 6139–6143.
- [8] Junteng Jia, Gil Keren, Wei Zhou, Egor Lakomkin, Xiaohui Zhang, Chunyang Wu, Frank Seide, Jay Mahadeokar, and Ozlem Kalinli, “Efficient Streaming LLM for Speech Recognition,” in *Proc. of ICASSP*, 2025, pp. 1–5.
- [9] Chung-Cheng Chiu and Colin Raffel, “Monotonic Chunkwise Attention,” in *Proc. of ICLR*, 2018.
- [10] Niko Moritz, Takaaki Hori, and Jonathan Le Roux, “Triggered Attention for End-to-end Speech Recognition,” in *Proc. of ICASSP*, 2019, pp. 5666–5670.
- [11] Emiru Tsunoo, Yosuke Kashiwagi, and Shinji Watanabe, “Streaming Transformer ASR With Blockwise Synchronous Beam Search,” in *Proc. of SLT*, 2020, pp. 22–29.
- [12] Mohammad Zeineldeen, Albert Zeyer, Ralf Schlüter, and Hermann Ney, “Chunked Attention-based Encoder-Decoder Model for Streaming Speech Recognition,” in *Proc. of ICASSP*, 2024, pp. 11331–11335.
- [13] Shinji Watanabe, Takaaki Hori, Suyoun Kim, John R. Hershey, and Tomoki Hayashi, “Hybrid CTC/Attention Architecture for End-to-End Speech Recognition,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 11, no. 8, pp. 1240–1253, 2017.
- [14] Shinji Watanabe, Takaaki Hori, Shigeki Karita, Tomoki Hayashi, Jiro Nishitoba, Yuya Unno, Nelson Enrique Yalta Solpin, Jahn Heymann, Matthew Wiesner, Nanxin Chen, et al., “ESPnet: End-to-End Speech Processing Toolkit,” 2018.
- [15] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur, “LibriSpeech: An ASR corpus based on public domain audio books,” in *Proc. of ICASSP*, 2015, pp. 5206–5210.
- [16] Kikuo Maekawa, Hanae Koiso, Sadaaki Furui, and Hitoshi Isahara, “Spontaneous Speech Corpus of Japanese,” in *Proc. of LREC*, 2000.
- [17] Tom Ko, Vijayaditya Peddinti, Daniel Povey, and Sanjeev Khudanpur, “Audio Augmentation for Speech Recognition,” in *Proc. of INTERSPEECH*, 2015, pp. 3586–3589.
- [18] Daniel S Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D Cubuk, and Quoc V Le, “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition,” *Proc. of INTERSPEECH*, p. 2613, 2019.
- [19] Daniel S Park, Yu Zhang, Chung-Cheng Chiu, Youzheng Chen, Bo Li, William Chan, Quoc V Le, and Yonghui Wu, “SpecAugment on Large Scale Datasets,” in *Proc. of ICASSP*, 2020, pp. 6879–6883.
- [20] Rico Sennrich, Barry Haddow, and Alexandra Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proc. of ACL*, 2016, pp. 1715–1725.
- [21] Michael McAuliffe, Michaela Socolof, Sarah Mihuc, Michael Wagner, and Morgan Sonderegger, “Montreal Forced Aligner: Trainable text-speech alignment using Kaldi,” in *Proc. of INTERSPEECH*, 2017, pp. 498–502.
- [22] Xie Chen, Yu Wu, Zhenghao Wang, Shujie Liu, and Jinyu Li, “Developing Real-Time Streaming Transformer Transducer for Speech Recognition on Large-Scale Dataset,” in *Proc. of ICASSP*, 2021, pp. 5904–5908.
- [23] Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [24] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber, “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks,” in *Proc. of ICML*, 2006, pp. 369–376.
- [25] Jaesong Lee and Shinji Watanabe, “Intermediate Loss Regularization for CTC-based Speech Recognition,” in *Proc. of ICASSP*, 2021, pp. 6224–6228.
- [26] Diederik P. Kingma and Jimmy Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. of ICLR*, 2014.
- [27] Golan Pundak and Tara N. Sainath, “Lower Frame Rate Neural Network Acoustic Models,” in *Proc. of INTERSPEECH*, 2016, pp. 22–26.
- [28] Gakuto Kurata and Kartik Audhkhasi, “Improved Knowledge Distillation from Bi-Directional to Uni-Directional LSTM CTC for End-to-End Speech Recognition,” in *Proc. of SLT*, 2018, pp. 411–417.
- [29] Takafumi Moriya, Masato Mimura, Kiyooki Matsui, Hiroshi Sato, and Kohei Matsuura, “Attention-Free Dual-Mode ASR with Latency-Controlled Selective State Spaces,” in *Proc. of INTERSPEECH*, 2025, pp. 3588–3592.
- [30] Takafumi Moriya, Masato Mimura, Tomohiro Tanaka, Hiroshi Sato, Ryo Masumura, and Atsunori Ogawa, “All-in-One ASR: Unifying Encoder-Decoder Models of CTC, Attention, and Transducer in Dual-Mode ASR,” in *Proc. of ASRU*, 2025.